Agent Harness 实战

- Agent Harness 实战：从 BaoClaw 看 AI Agent 系统架构
  - 关于本书
  - 参考实现：BaoClaw
  - 目录
  - 写作理念
  - License
- 第 1 章：Agent 的本质 —— 从 LLM 到 Agent
  - 1.1 LLM 不是 Agent
  - 1.2 Agent 的三个核心能力
  - 1.3 Agent Harness 是什么
  - 1.4 BaoClaw 的整体架构
  - 1.5 本书的路线图
- 第 2 章：ReAct 循环 —— Agent 的心跳
  - 2.1 什么是 ReAct
  - 2.2 BaoClaw 的 ReAct 实现
  - 2.3 事件驱动的架构
  - 2.4 与其他框架的对比
  - 2.5 小结
- 第 3 章：Agent Harness 架构概览
  - 3.1 为什么需要 Harness
  - 3.2 BaoClaw 的三层架构
  - 3.3 Daemon 模式
  - 3.4 数据流
  - 3.5 配置体系
  - 3.6 小结
- 第 4 章：工具系统设计 —— Tool Trait 与执行器
  - 4.1 问题：Agent 如何获得行动能力
  - 4.2 模式：Tool Trait
  - 4.3 内置工具
  - 4.4 工具执行器
  - 4.5 小结
- 第 5 章：MCP 协议 —— 让 Agent 获得无限能力
  - 5.1 问题：内置工具的局限

    - 5.2 MCP 协议概述
    - 5.3 BaoClaw 的 MCP 实现
    - 5.4 配置
    - 5.5 踩过的坑
    - 5.6 小结
- 第 6 章：Skills —— 可插拔的 Agent 行为
    - 6.1 问题：工具 vs 指令
    - 6.2 模式：Skill = 系统提示词注入
    - 6.3 BaoClaw 的实现
    - 6.4 Skill vs Tool 的选择
    - 6.5 小结
- 第 7 章：Plugins —— 打包分发的能力套件
    - 7.1 问题：能力的组合与分发
    - 7.2 模式：Plugin = Skill + MCP + Tools 的容器
    - 7.3 BaoClaw 的实现
    - 7.4 小结
- 第 8 章：上下文管理 —— 系统提示词的构建
    - 8.1 问题：AI 需要知道什么
    - 8.2 BaoClaw 的系统提示词构建
    - 8.3 BAOCLAW.md —— 项目级指令
    - 8.4 上下文窗口预算
    - 8.5 小结
- 第 9 章：短期记忆 —— 对话历史与 Compact
    - 9.1 问题：对话越来越长怎么办
    - 9.2 BaoClaw 的短期记忆
    - 9.3 Compact 机制
    - 9.4 小结
- 第 10 章：长期记忆 —— 跨会话的知识持久化
    - 10.1 问题：Agent 的“失忆症”
    - 10.2 模式：JSONL 文件 + 系统提示词注入
    - 10.3 BaoClaw 的实现
    - 10.4 两种写入方式
    - 10.5 小结
- 第 11 章：会话设计 —— 共享、恢复与多客户端
    - 11.1 问题：多个入口，一个大脑
    - 11.2 会话模式

- 11.3 SharedSession 的设计（`claude-core/src/engine/shared_session.rs`）
- 11.4 会话持久化（Transcript）
- 11.5 BaoClaw 的客户端连接流程
- 11.6 小结

- 第 12 章：守护进程架构 —— Daemon 模式
  - 12.1 问题：Agent 进程的生命周期
  - 12.2 模式：Daemon + 客户端分离
  - 12.3 小结
- 第 13 章：IPC 协议 —— JSON-RPC over UDS
  - 13.1 协议选择
  - 13.2 消息类型
  - 13.3 方法列表（`claude-core/src/ipc/router.rs`）
  - 13.4 流式事件
  - 13.5 小结
- 第 14 章：多客户端接入 —— 终端、Telegram、更多
  - 14.1 客户端是“薄”的
  - 14.2 终端 CLI（`ts-ipc/cli.ts`）
  - 14.3 Telegram 网关（`baoclaw-telegram/src/gateway.ts`）
  - 14.4 添加新客户端
  - 14.5 小结
- 第 15 章：共享会话 —— SharedQueryEngine 的设计
  - 15.1 问题：从“各自独立”到“实时共享”
  - 15.2 设计：SessionRegistry + SharedSession
  - 15.3 并发控制的三个层次
  - 15.4 事件广播
  - 15.5 小结
- 第 16 章：错误处理与恢复 —— 从 Fallback 到自动 Compact
  - 16.1 问题：生产环境中什么都会出错
  - 16.2 错误分类与处理策略
  - 16.3 设计原则
  - 16.4 小结
- 第 17 章：流式输出 —— SSE 事件与广播
  - 17.1 问题：用户不想等
  - 17.2 两层流式架构
  - 17.3 事件类型（`EngineEvent`）
  - 17.4 共享模式下的广播

- 17.5 客户端渲染
- 17.6 小结
- 第 18 章：权限控制 —— PermissionGate 模式
  - 18.1 问题：AI 不应该无限制地行动
  - 18.2 模式：请求-授权-执行
  - 18.3 BaoClaw 的实现
  - 18.4 哪些工具需要授权
  - 18.5 小结
- 第 19 章：成本追踪 —— Token 计量与预算
  - 19.1 问题：AI 调用不是免费的
  - 19.2 BaoClaw 的成本追踪
  - 19.3 成本优化策略
  - 19.4 小结
- 第 20 章：Computer Use —— 桌面控制 Agent
  - 20.1 从代码到桌面
  - 20.2 BaoClaw 的 Computer Use 实现
  - 20.3 图片处理的挑战
  - 20.4 工作流示例
  - 20.5 小结
- 第 22 章：多模型支持 —— Fallback 与模型切换
  - 22.1 问题：不要把鸡蛋放在一个篮子里
  - 22.2 BaoClaw 的多模型架构
  - 22.3 兼容不同 API
  - 22.4 小结
- 附录 A：BaoClaw 完整架构图
  - 系统架构
  - 文件系统布局
- 附录 B：从零搭建 Agent Harness
  - 最小可行 Agent
  - 技术选型建议
- 附录 C：与其他框架的对比
  - Agent Harness 框架对比
  - 选型建议
  - BaoClaw 的独特之处

Agent Harness 实战：从 BaoClaw 看 AI

Agent 系统架构

> 以一个真实的 Agent Harness 产品为原型，深入剖析 AI Agent 系统的设计模式与工程实践。

关于本书

本书以 BaoClaw —— 一个基于 Rust + TypeScript 构建的 AI 编码助手 —— 作为参考实现，系统讲解 Agent Harness（Agent 运行时框架）的架构设计、核心模式和生产实践。

与市面上大多数 Agent 教程不同，本书不是 API 调用指南或 Prompt 技巧集锦，而是从工程角度回答：

- 一个 Agent 系统的核心循环是什么？
- 工具系统如何设计才能可扩展？
- 多客户端如何共享同一个 Agent 会话？
- 记忆系统如何跨会话持久化？
- MCP 协议如何让 Agent 获得无限能力？
- 生产环境中如何处理上下文窗口、错误恢复、并发控制？

每一个问题，都有 BaoClaw 的真实代码作为答案。

参考实现：BaoClaw

BaoClaw 是一个完整的 Agent Harness 系统，包含：

组件	技术栈	职责
claude-core	Rust	守护进程、QueryEngine、工具执行、IPC 服务
ts-ipc	TypeScript	终端 CLI 客户端
baoclaw-telegram	TypeScript	Telegram 网关客户端

目录

Part 1：Agent 基础

- 第 1 章：Agent 的本质 —— 从 LLM 到 Agent
- 第 2 章：ReAct 循环 —— Agent 的心跳
- 第 3 章：Agent Harness 架构概览

Part 2：工具与扩展

- 第 4 章：工具系统设计 —— Tool Trait 与执行器
- 第 5 章：MCP 协议 —— 让 Agent 获得无限能力
- 第 6 章：Skills —— 可插拔的 Agent 行为
- 第 7 章：Plugins —— 打包分发的能力套件

Part 3：上下文与记忆

- 第 8 章：上下文管理 —— 系统提示词的构建
- 第 9 章：短期记忆 —— 对话历史与 Compact
- 第 10 章：长期记忆 —— 跨会话的知识持久化
- 第 11 章：会话设计 —— 共享、恢复与多客户端

Part 4：IPC 与多客户端

- 第 12 章：守护进程架构 —— Daemon 模式
- 第 13 章：IPC 协议 —— JSON-RPC over UDS
- 第 14 章：多客户端接入 —— 终端、Telegram、更多
- 第 15 章：共享会话 —— SharedQueryEngine 的设计

Part 5：生产实践

- 第 16 章：错误处理与恢复 —— 从 Fallback 到自动 Compact
- 第 17 章：流式输出 —— SSE 事件与广播
- 第 18 章：权限控制 —— PermissionGate 模式
- 第 19 章：成本追踪 —— Token 计量与预算

Part 6：高级模式

- 第 20 章：Computer Use —— 桌面控制 Agent
- 第 21 章：Agentic Coding —— 代码生成与编辑
- 第 22 章：多模型支持 —— Fallback 与模型切换

附录

- 附录 A：BaoClaw 完整架构图
- 附录 B：从零搭建 Agent Harness
- 附录 C：与其他框架的对比

写作理念

模式优先，代码为证。

每一章遵循这样的结构： 1. **问题** —— 什么场景需要这个能力？ 2. **模式** —— 通用的设计模式是什么？ 3. **实现** —— BaoClaw 是怎么做的？（附真实代码） 4. **思考** —— 还有哪些替代方案？

License

本书内容采用 CC BY-NC-SA 4.0 协议。

第 1 章：Agent 的本质 —— 从 LLM 到 Agent

1.1 LLM 不是 Agent

大语言模型（LLM）本质上是一个函数：输入一段文本，输出一段文本。它没有记忆、没有工具、没有自主行动的能力。你问它“帮我创建一个文件”，它只能告诉你怎么做，但不能真的去做。

```
LLM: text → text
```

Agent 不同。Agent 是一个**循环系统**：它接收用户指令，思考需要做什么，调用工具执行动作，观察结果，然后决定下一步。这个循环一直持续，直到任务完成。

```
Agent: 指令 → [思考 → 行动 → 观察]* → 结果
```

关键区别在于：LLM 是一次性的输入输出，Agent 是持续的循环决策。

1.2 Agent 的三个核心能力

一个 Agent 系统需要三个核心能力：

1. 推理（Reasoning）

Agent 需要理解用户意图，分解任务，制定计划。这部分由 LLM 提供。

2. 行动（Action）

Agent 需要能够执行动作 —— 读写文件、运行命令、调用 API。这部分由工具系统（Tool System）提供。

3. 记忆（Memory）

Agent 需要记住之前发生了什么 —— 对话历史、用户偏好、项目上下文。这部分由记忆系统（Memory System）提供。

在 BaoClaw 中，这三个能力分别对应：

能力	BaoClaw 实现	代码位置
推理	QueryEngine 调用 LLM API	`claude-core/src/engine/query_engine.rs`
行动	Tool trait + 内置工具 + MCP	`claude-core/src/tools/`
记忆	Messages + Transcript + MemoryStore	`claude-core/src/engine/`

1.3 Agent Harness 是什么

Agent Harness（Agent 运行时框架）是承载 Agent 运行的基础设施。它不是 Agent 本身，而是让 Agent 能够运行的“容器”。

类比： - LLM 是引擎 - Agent 是驾驶员 - Agent Harness 是整辆车 —— 包括方向盘、油门、刹车、仪表盘

BaoClaw 就是一个 Agent Harness。它提供：

- **进程管理**：Daemon 模式，后台运行
- **IPC 通信**：JSON-RPC over Unix Domain Socket
- **工具注册**：Tool trait，统一的工具接口
- **上下文构建**：系统提示词、Skills、Memory 的组装
- **流式输出**：SSE 事件流，实时反馈
- **多客户端**：终端、Telegram、未来更多
- **会话管理**：共享会话、会话恢复、Compact

1.4 BaoClaw 的整体架构

```
┌──────────────────────────────────────────────────────────────┐
│                    Clients                                   │
│   ┌──────────┐   ┌──────────────┐   ┌──────────────┐        │
│   │ Terminal │   │   Telegram   │   │  Future      │        │
│   │  CLI     │   │   Gateway    │   │  Clients     │        │
│   └────┬─────┘   └──────┬───────┘   └──────┬───────┘        │
│        │                │                  │                │
│        └────────────────┴──────────────────┘                │
│                         │ IPC (JSON-RPC / UDS)              │
│                         ▼                                   │
│   ┌──────────────────────────────────────────────────────┐  │
│   │              Daemon (Rust)                           │  │
│   │   ┌──────────────────────────────────────────┐       │  │
│   │   │         SharedState                      │       │  │
│   │   │   ┌──────────┐   ┌──────────────────┐    │       │  │
│   │   │   │ Tools    │   │ SessionRegistry  │    │       │  │
│   │   │   │ (builtin │   │ (shared mode)    │    │       │  │
│   │   │   │  + MCP)  │   └──────────────────┘    │       │  │
│   │   │   └──────────┘   ┌──────────────────┐    │       │  │
│   │   │   ┌──────────┐   │ MemoryStore      │    │       │  │
│   │   │   │ Skills   │   │ (long-term)      │    │       │  │
│   │   │   └──────────┘   └──────────────────┘    │       │  │
│   │   └──────────────────────────────────────────┘       │  │
│   │                     │                                │  │
│   │                     ▼                                │  │
│   │   ┌──────────────────────────────────────────┐       │  │
│   │   │         QueryEngine                      │       │  │
│   │   │  Messages ←→ LLM API ←→ Tools            │       │  │
│   │   │  (ReAct Loop)                            │       │  │
│   │   └──────────────────────────────────────────┘       │  │
│   └──────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

1.5 本书的路线图

接下来的章节将逐层深入这个架构：

- **Part 1**（本部分）：理解 Agent 的核心概念和 ReAct 循环
- **Part 2**：深入工具系统 —— 从内置工具到 MCP 协议
- **Part 3**：上下文与记忆 —— 短期、长期、项目级
- **Part 4**：IPC 与多客户端 —— Daemon 架构和共享会话
- **Part 5**：生产实践 —— 错误处理、流式输出、权限控制
- **Part 6**：高级模式 —— Computer Use、Agentic Coding

每一章都会指向 BaoClaw 的真实代码，让你不仅理解“为什么”，还能看到“怎么做”。

第 2 章：ReAct 循环 —— Agent 的心跳

2.1 什么是 ReAct

ReAct（Reasoning + Acting）是 Agent 的核心运行模式。它的本质是一个循环：

```
用户消息 → LLM 思考 → 决定行动 → 执行工具 → 观察结果 → LLM 再思考 → ... → 最终回复
```

这个循环是 Agent 的“心跳”。没有它，LLM 只能做一次性的问答；有了它，Agent 可以完成任意复杂的多步骤任务。

2.2 BaoClaw 的 ReAct 实现

在 BaoClaw 中，ReAct 循环实现在 `run_query_loop` 函数中（`claude-core/src/engine/query_engine.rs`）。

核心流程：

```
loop {
    // 1. 构建 API 请求（包含完整对话历史 + 工具定义）
    let request = build_api_request(&messages, &config);

    // 2. 调用 LLM API（流式）
    let mut stream = api_client.create_message_stream(request).await?;

    // 3. 处理流式响应，收集 content blocks
    //    - TextBlock: AI 的文字回复
    //    - ToolUseBlock: AI 决定调用的工具
    //    - ThinkingBlock: 扩展思考内容
    while let Some(event) = stream.next().await { ... }

    // 4. 检查是否有工具调用
    let tool_uses = extract_tool_uses(&assistant_content_blocks);

    if tool_uses.is_empty() {
        // 没有工具调用 → 任务完成，返回结果
        return;
    }

    // 5. 执行工具
    let tool_results = execute_tools(&tools, &tool_uses, &context).await;

    // 6. 将工具结果加入对话历史
    messages.push(build_tool_result_message(&tool_results));

    // 7. 回到步骤 1，让 LLM 看到工具结果后继续思考
    turn_count += 1;
}
```

关键设计点：

2.2.1 流式处理

BaoClaw 使用 SSE（Server-Sent Events）流式接收 LLM 的响应。每收到一个 token，就通过事件通道发送给客户端，实现实时打字效果：

```
ApiStreamEvent::ContentBlockDelta { delta, .. } => {
    if let Some(text) = delta.get("text").and_then(|v| v.as_str()) {
        // 实时发送给客户端
        tx.send(EngineEvent::AssistantChunk { content: text.to_string() }).await;
    }
}
```

2.2.2 工具调用的闭环

当 LLM 决定调用工具时，它会在响应中包含一个 `tool_use` content block：

```
{
    "type": "tool_use",
    "id": "tu_001",
    "name": "Bash",
    "input": { "command": "ls -la" }
}
```

BaoClaw 提取这些 tool_use blocks，执行对应的工具，然后将结果作为 `tool_result` 消息追加到对话历史中。LLM 在下一轮看到工具结果后，可以继续调用更多工具或给出最终回复。

2.2.3 终止条件

循环在以下情况终止：

1. **LLM 不再调用工具** —— `tool_uses.is_empty()`，任务完成
2. **达到最大轮次** —— `turn_count >= max_turns`，防止无限循环
3. **用户中止** —— `abort` 信号被触发
4. **API 错误** —— 网络错误、认证失败等
5. **上下文窗口超出** —— 自动 compact 后重试

2.3 事件驱动的架构

BaoClaw 的 ReAct 循环不是简单的请求-响应，而是事件驱动的。`QueryEngine` 通过 `mpsc::channel` 向客户端发送各种事件：

```
pub enum EngineEvent {
    AssistantChunk { content: String },      // AI 输出的文字片段
    ThinkingChunk { content: String },       // 扩展思考内容
    ToolUse { tool_name, input, id },        // AI 决定调用工具
    ToolResult { id, output, is_error },     // 工具执行结果
    PermissionRequest { tool_name, input }, // 需要用户授权
    Result(QueryResult),                     // 最终结果
    Error(EngineError),                      // 错误
    ModelFallback { from, to },              // 模型降级
    StateUpdate { patch },                   // 状态更新
}
```

这种设计让客户端（终端 CLI、Telegram）可以实时展示 Agent 的每一步动作，而不是等到全部完成才返回。

2.4 与其他框架的对比

特性	BaoClaw	LangChain	Claude Code
循环实现	Rust async loop	Python chain	TypeScript loop
流式输出	SSE events via IPC	Callbacks	Direct stdout
工具执行	并行 + 权限控制	串行	并行
错误恢复	Fallback + auto-compact	手动重试	手动重试

2.5 小结

ReAct 循环是 Agent 的核心。理解了这个循环，你就理解了 Agent 系统的骨架。接下来的章节将深入循环中的每个组件 —— 工具系统、上下文管理、记忆系统 —— 看它们如何协同工作。

第 3 章：Agent Harness 架构概览

3.1 为什么需要 Harness

直接调用 LLM API 写一个 chatbot 很简单。但要构建一个生产级的 Agent 系统，你需要解决大量“API 之外”的问题：

- 工具怎么注册和发现？
- 多个客户端怎么连接同一个 Agent？
- 对话历史太长怎么办？
- 工具执行需要用户授权怎么处理？
- Agent 进程崩溃了怎么恢复？

Agent Harness 就是解决这些问题的框架。它是 Agent 的“运行时环境”。

3.2 BaoClaw 的三层架构

BaoClaw 采用经典的三层架构：

```
┌──────────────────────────────────────┐
│        Client Layer                  │  TypeScript
│  (CLI, Telegram, ...)                │
├──────────────────────────────────────┤
│        IPC Layer                     │  JSON-RPC 2.0 / UDS
│  (Protocol, Routing)                 │
├──────────────────────────────────────┤
│        Core Layer                    │  Rust
│  (QueryEngine, Tools, Memory)        │
└──────────────────────────────────────┘
```

Client Layer（客户端层）

负责用户交互。每个客户端是一个独立进程，通过 IPC 连接到 Core。

- `ts-ipc/cli.ts` —— 终端 CLI，富文本 TUI
- `baoclaw-telegram/src/gateway.ts` —— Telegram Bot 网关

客户端是“薄”的 —— 只负责 UI 渲染和用户输入，不包含业务逻辑。

IPC Layer（通信层）

客户端和 Core 之间的通信协议。

- 传输：Unix Domain Socket（UDS）
- 协议：JSON-RPC 2.0 + NDJSON 分帧
- 发现：`/tmp/baoclaw-sockets/` 目录下的 metadata JSON 文件

Core Layer（核心层）

Agent 的大脑。一个 Rust 守护进程，包含：

组件	职责
QueryEngine	ReAct 循环、LLM API 调用
Tool System	工具注册、执行、权限控制
MemoryStore	长期记忆持久化
SessionRegistry	共享会话管理
TranscriptWriter	对话记录持久化
PermissionGate	工具执行授权
CostTracker	Token 计量和成本追踪
TaskManager	后台任务管理

3.3 Daemon 模式

BaoClaw 的 Core 以 Daemon（守护进程）模式运行：

```
# CLI 启动时自动 spawn daemon
baoclaw
  └→ spawn claude-core --daemon --cwd /path/to/project
       └→ 监听 UDS socket
       └→ 输出 SOCKET:/tmp/baoclaw-sockets/baoclaw-xxx.sock
       └→ 写入 metadata JSON
       └→ 等待客户端连接
```

Daemon 模式的优势：

1. **进程独立** —— CLI 退出不影响 daemon，Telegram 可以随时连接
2. **多客户端** —— 多个客户端可以同时连接同一个 daemon
3. **会话持久** —— 对话历史在 daemon 进程内存中，不随客户端断开而丢失
4. **资源共享** —— MCP 服务器连接、工具实例在所有客户端间共享

3.4 数据流

一次完整的用户交互的数据流：

```
1. 用户在终端输入 "帮我创建一个 hello.py"
2. CLI 发送 JSON-RPC: submitMessage { prompt: "帮我创建一个 hello.py" }
3. Daemon 收到请求，QueryEngine 开始 ReAct 循环
4. QueryEngine 调用 LLM API（带系统提示词 + 对话历史 + 工具定义）
5. LLM 返回 tool_use: FileWrite { path: "hello.py", content: "..." }
6. Daemon 发送 stream/event: tool_use 通知给客户端
7. QueryEngine 执行 FileWriteTool
8. Daemon 发送 stream/event: tool_result 通知给客户端
9. QueryEngine 再次调用 LLM（带工具结果）
10. LLM 返回文字回复 "已创建 hello.py"
11. Daemon 发送 stream/event: assistant_chunk 通知给客户端
12. Daemon 发送 stream/event: result 通知给客户端
13. CLI 渲染最终结果
```

如果 Telegram 也连接了同一个 daemon（共享模式），步骤 6-12 的事件会同时广播给 Telegram 客户端。

3.5 配置体系

BaoClaw 的配置分为多个层级：

层级	文件	作用域
全局配置	`~/.baoclaw/config.json`	所有项目
全局 MCP	`~/.baoclaw/mcp.json`	所有项目
全局 Skills	`~/.baoclaw/skills/`	所有项目
全局 Memory	`~/.baoclaw/memory.jsonl`	所有项目
项目配置	`<cwd>/.baoclaw/`	当前项目
项目指令	`<cwd>/BAOCLAW.md`	当前项目
项目 MCP	`<cwd>/.baoclaw/mcp.json`	当前项目
项目 Skills	`<cwd>/.baoclaw/skills/`	当前项目

这种分层设计让用户可以在全局设置通用偏好，在项目级设置特定规则。

3.6 小结

Agent Harness 的核心职责是：让 Agent 能够稳定、高效、安全地运行。BaoClaw 通过 Daemon 模式 + IPC 通信 + 分层配置实现了这一目标。接下来我们将深入每个子系统的设计细节。

第 4 章：工具系统设计 —— Tool Trait 与执行器

4.1 问题：Agent 如何获得行动能力

LLM 只能生成文本。要让 Agent 能读写文件、运行命令、搜索网页，需要一个工具系统。

核心问题： - 如何定义统一的工具接口？ - 如何让 LLM 知道有哪些工具可用？ - 如何安全地执行工具？

4.2 模式：Tool Trait

BaoClaw 使用 Rust 的 trait 定义统一的工具接口（`claude-core/src/tools/trait_def.rs`）：

```
#[async_trait]
pub trait Tool: Send + Sync {
    fn name(&self) -> &str;                     // 工具名称
    fn input_schema(&self) -> JsonSchema;       // 输入参数的 JSON Schema
    fn prompt(&self) -> String;                 // 给 LLM 看的工具描述

    async fn call(
        &self,
        input: Value,                           // LLM 提供的参数
        context: &ToolContext,                  // 执行上下文（cwd, model,
  abort_signal）
        progress: &dyn ProgressSender,          // 进度回调
    ) -> Result<ToolResult, ToolError>;

    fn max_result_size_chars(&self) -> usize { 100_000 }  // 结果截断阈值
    fn is_read_only(&self, input: &Value) -> bool { false }
}
```

这个设计的关键点：

1. `input_schema` —— 返回 JSON Schema，告诉 LLM 这个工具接受什么参数。LLM 会根据 schema 生成正确的 JSON 输入。
2. `prompt` —— 工具的自然语言描述，帮助 LLM 理解什么时候该用这个工具。
3. `Send + Sync` —— 工具实例在多个异步任务间共享（`Arc<dyn Tool>`）。

4. `max_result_size_chars` —— 防止工具返回过大的结果撑爆上下文窗口。MCP 工具覆盖为 10MB 以支持截图。

4.3 内置工具

BaoClaw 内置了 14 个工具：

工具	职责
BashTool	执行 shell 命令
FileReadTool	读取文件内容
FileWriteTool	创建/覆盖文件
FileEditTool	编辑文件（搜索替换）
GrepTool	正则搜索文件内容
GlobTool	文件路径模式匹配
WebFetchTool	获取网页内容
WebSearchTool	网页搜索
AgentTool	子 Agent（只读工具集）
TodoWriteTool	TODO 列表管理
NotebookEditTool	Jupyter Notebook 编辑
ToolSearchTool	搜索可用工具
MemoryTool	自动保存长期记忆
ProjectNoteTool	自动写入项目规则

4.4 工具执行器

工具执行在 `claude-core/src/tools/executor.rs` 中实现：

```rust
pub async fn execute_tools(
    tools: &[Arc<dyn Tool>],
    tool_uses: &[ToolUseRequest],
    context: &ToolContext,
    progress: &dyn ProgressSender,
) -> Vec<ToolExecutionResult> {
    // 并行执行所有工具调用
    let futures: Vec<_> = tool_uses.iter().map(|tu| {
        let tool = find_tool(tools, &tu.name);
        async move {
            match tool {
                Some(t) => {
                    let result = t.call(tu.input.clone(), context, progress).await;
                    // 截断过大的结果
                    let output = truncate_if_needed(result.data,
  t.max_result_size_chars());
                    ToolExecutionResult { tool_use_id, output, is_error }
                }
                None => ToolExecutionResult {
                    output: "Tool not found", is_error: true
                }
            }
        }
    }).collect();

    futures::future::join_all(futures).await
}
```

关键设计：**结果截断**。`truncate_if_needed` 确保工具返回的数据不会超过阈值，防止一张截图的 base64 数据撑爆 200K 的上下文窗口。

4.5 小结

工具系统是 Agent 的“手脚”。统一的 Tool trait 让内置工具和 MCP 外部工具使用相同的接口，执行器负责并行执行和结果截断。下一章我们将看到 MCP 协议如何让 Agent 获得无限的外部工具。

第 5 章：MCP 协议 —— 让 Agent 获得无限能力

5.1 问题：内置工具的局限

内置工具是有限的。你不可能为每个场景都写一个 Rust 工具。用户可能需要： - 控制桌面（截图、点击） - 操作数据库 - 调用企业内部 API - 连接智能家居设备

MCP（Model Context Protocol）解决了这个问题：它定义了一个标准协议，让任何外部程序都能作为工具服务器接入 Agent。

5.2 MCP 协议概述

MCP 使用 JSON-RPC 2.0 over stdio 通信：

```
Agent (Client)                    MCP Server (外部进程)
    │                                     │
    │── initialize ──────────────────────>│
    │<─ capabilities, serverInfo ─────────│
    │── notifications/initialized ───────>│
    │── tools/list ──────────────────────>│
    │<─ { tools: [...] } ─────────────────│
    │                                     │
    │── tools/call { name, arguments } ──>│
    │<─ { content: [...] } ───────────────│
```

5.3 BaoClaw 的 MCP 实现

传输层（`claude-core/src/mcp/transport.rs`）

`StdioTransport` 负责 spawn 子进程并通过 stdin/stdout 通信：

```rust
pub async fn spawn(command: &str, args: &[String], env: &HashMap<String, String>)
    -> Result<Self, McpError>
{
    let mut child = Command::new(command).args(args).envs(env)
        .stdin(Stdio::piped()).stdout(Stdio::piped()).stderr(Stdio::piped())
        .spawn()?;

    // MCP initialize 握手
    transport.request("initialize", Some(json!({
        "protocolVersion": "2024-11-05",
        "capabilities": {},
        "clientInfo": { "name": "baoclaw", "version": "0.3.0" }
    }))).await?;

    // 发送 initialized 通知
    transport.notify("notifications/initialized", None).await?;

    Ok(transport)
}
```

工具发现（`claude-core/src/mcp/client.rs`）

连接成功后，调用 `tools/list` 获取工具列表：

```rust
pub async fn refresh_tools(&self) -> Result<(), McpError> {
    let result = transport.request("tools/list", None).await?;
    let tool_defs: Vec<McpToolDef> =
  serde_json::from_value(result["tools"].clone())?;
    *self.tools.write().await = tool_defs;
}
```

注意：MCP 返回的字段名是 camelCase（`inputSchema`），而 Rust 用 snake_case。需要 `#[serde(rename = "inputSchema")]`。

工具包装（`claude-core/src/mcp/tool_wrapper.rs`）

`McpToolWrapper` 将 MCP 工具包装为 BaoClaw 的 `Tool` trait：

```rust
impl Tool for McpToolWrapper {
    fn name(&self) -> &str { &self.tool_def.name }
    fn prompt(&self) -> String {
        format!("[MCP:{}] {}", self.server_name, self.tool_def.description)
    }
    fn max_result_size_chars(&self) -> usize { 10_000_000 } // 10MB，支持截图

    async fn call(&self, input: Value, ...) -> Result<ToolResult, ToolError> {
        self.client.call_tool(&self.tool_def.name, input).await
    }
}
```

5.4 配置

MCP 服务器在 `~/.baoclaw/mcp.json` 中配置：

```json
{
    "mcpServers": {
        "computer-control-mcp": {
            "command": "uvx",
            "args": ["computer-control-mcp@latest"]
        }
    }
}
```

BaoClaw 在 daemon 启动时自动发现、连接所有配置的 MCP 服务器，并将它们的工具注册到 `engine_tools` 中。

5.5 踩过的坑

1. **字段名不匹配** —— MCP 用 `inputSchema`（camelCase），Rust struct 默认 snake_case，导致所有工具解析失败返回 0 个。解决：`#[serde(rename)]`。
2. **结果截断** —— 截图工具返回的 base64 数据超过 100K 字符被截断，JSON 不完整。解决：MCP 工具覆盖 `max_result_size_chars` 为 10MB。
3. **base64 图片在对话历史中** —— 一张 2560×1600 截图的 base64 约 5MB，直接撑爆上下文窗口。解决：`strip_base64_images` 在存入对话历史前移除图片数据。

5.6 小结

MCP 是 Agent 能力扩展的关键协议。通过标准化的 JSON-RPC 接口，任何外部程序都能成为 Agent 的工具。BaoClaw 通过 `McpToolWrapper` 将 MCP 工具无缝集成到内置工具系统中。

第 6 章：Skills —— 可插拔的 Agent 行为

6.1 问题：工具 vs 指令

工具（Tool）给 Agent 行动能力，但有些能力不是“调用一个函数”能解决的。比如：

- “用 xdotool 截图分析桌面图标然后点击” —— 这是一个多步骤的工作流
- “模拟一场辩论，正方反方交替发言” —— 这是一种对话模式

这些不是单个工具调用，而是**行为指令** —— 告诉 Agent “遇到这种场景时，按这个流程做”。

6.2 模式：Skill = 系统提示词注入

BaoClaw 的 Skill 本质上是一段 Markdown 文本，在 daemon 启动时被注入到系统提示词中。AI 在每次对话时都能看到这些指令。

Skill 文件格式（`~/.baoclaw/skills/desktop-launch-app/SKILL.md`）：

```
---
name: desktop-launch-app
description: 在桌面上通过开始菜单启动应用程序
allowed-tools:
  - Bash
  - FileWrite
---

# 通过开始菜单启动桌面应用

通过视觉截图分析 + 鼠标点击的方式，在桌面环境中打开开始菜单并启动指定应用程序。

## 步骤
1. 使用 xdotool 点击左下角启动器
2. 等待菜单出现
3. 截图分析菜单内容
4. 找到目标应用图标并点击
```

YAML frontmatter 包含元数据（名称、描述、允许的工具），Markdown body 包含详细指令。

6.3 BaoClaw 的实现

发现（`claude-core/src/discovery/skills.rs`）

```rust
pub async fn discover_skills(cwd: &Path) -> Vec<SkillInfo> {
    // 扫描三个位置:
    // 1. 项目级: <cwd>/.baoclaw/skills/
    // 2. 用户级: ~/.baoclaw/skills/
    // 3. 插件级: ~/.baoclaw/plugins/*/skills/
}
```

支持两种文件格式：- 目录格式：`skill-name/SKILL.md` - 单文件格式：`skill-name.md`

加载与注入

```rust
pub async fn load_skills_for_prompt(cwd: &Path) -> Option<String> {
    let skills = discover_skills(cwd).await;
    if skills.is_empty() { return None; }

    let mut parts = vec!["# Loaded Skills\n\n...".to_string()];
    for skill in &skills {
        let content = fs::read_to_string(&skill.path).await?;
        parts.push(format!("## Skill: {} [source: {}]\n\n{}",
            skill.name, skill.source, content));
    }
    Some(parts.join("\n"))
}
```

加载后的内容通过 `append_system_prompt` 注入到 `build_system_prompt` 中，AI 在每次 API 调用时都能看到。

6.4 Skill vs Tool 的选择

维度	Skill	Tool
本质	文本指令	可执行函数
注入方式	系统提示词	工具列表
消耗	占用上下文窗口	不占用（直到被调用）
适用场景	多步骤工作流、对话模式	单一原子操作
创建难度	写 Markdown	写 Rust 代码

6.5 小结

Skill 是 Agent 的“说明书”，通过系统提示词注入让 AI 学会新的行为模式。它的优势是零代码——任何人都可以用 Markdown 创建新的 Skill。

第 7 章：Plugins —— 打包分发的能力套件

7.1 问题：能力的组合与分发

一个完整的 Agent 能力通常需要多个组件配合： - 一个 MCP 服务器提供底层工具 - 一个 Skill 提供使用指令 - 可能还需要配置文件

如何把这些打包成一个可分发的单元？

7.2 模式：Plugin = Skill + MCP + Tools 的容器

BaoClaw 的 Plugin 是一个目录，可以包含：

```
~/.baoclaw/plugins/computer-control/
├── plugin.json          # 清单文件
├── skills/              # Skill 文件
│   └── desktop-launch/
│       └── SKILL.md
├── mcp.json             # MCP 服务器配置
└── tools/               # 自定义工具（未来）
```

plugin.json 清单

```
{
    "name": "computer-control",
    "version": "1.0.0",
    "description": "桌面控制能力套件"
}
```

7.3 BaoClaw 的实现

发现（`claude-core/src/discovery/plugins.rs`）

```rust
pub async fn discover_plugins(cwd: &Path) -> Vec<PluginInfo> {
    // 扫描 ~/.baoclaw/plugins/ 和 <cwd>/.baoclaw/plugins/
    // 检测子目录中的 plugin.json、skills/、mcp.json、tools/
}
```

加载

Plugin 的加载分散在各个发现模块中：

- **Skills 加载**：`discover_skills` 会扫描 `plugins/*/skills/` 目录
- **MCP 加载**：`discover_mcp_servers` 会扫描 `plugins/*/mcp.json`
- **Tools 加载**：暂未实现（需要动态加载机制）

来源标记为 `plugin:<name>`，方便用户区分。

7.4 小结

Plugin 是能力的打包单元。通过标准的目录结构，一个 Plugin 可以同时提供 Skill 指令和 MCP 工具，实现“安装即可用”的体验。

第 8 章：上下文管理 —— 系统提示词的构建

8.1 问题：AI 需要知道什么

每次 API 调用，AI 看到的信息由三部分组成： 1. **系统提示词（System Prompt）** —— AI 的“身份”和“规则” 2. **对话历史（Messages）** —— 之前的对话内容 3. **工具定义（Tools）** —— 可用的工具列表

系统提示词是最重要的 —— 它决定了 AI 的行为方式。

8.2 BaoClaw 的系统提示词构建

`build_system_prompt` （ `query_engine.rs` ）按顺序组装多个片段：

```
pub fn build_system_prompt(config: &QueryLoopConfig) -> Option<Vec<Value>> {
    let mut parts: Vec<String> = Vec::new();

    // 1. 基础身份
    parts.push("You are a helpful AI coding assistant.");

    // 2. 项目指令（BAOCLAW.md）
    if let Some(instructions) = &config.project_instructions {
        parts.push(format!("# Project Instructions\n\n{}", instructions));
    }

    // 3. Git 状态
    if let Some(git_info) = &config.git_info {
        parts.push(format!("# Git Status\n\nBranch: {}\n...", git_info.branch));
    }

    // 4. Skills + 长期记忆（通过 append_system_prompt）
    if let Some(append) = &config.append_system_prompt {
        parts.push(append.clone());
    }

    Some(vec![json!({ "type": "text", "text": parts.join("\n\n") })])
}
```

各层内容来源

层	来源	加载时机
基础身份	硬编码	每次调用
项目指令	`<cwd>/BAOCLAW.md`	每次调用时读取
Git 状态	`git status`	每次调用时获取
Skills	`~/.baoclaw/skills/`	daemon 启动时加载
长期记忆	`~/.baoclaw/memory.jsonl`	daemon 启动时加载

8.3 BAOCLAW.md —— 项目级指令

类似 Claude Code 的 `CLAUDE.md`，BaoClaw 支持项目级指令文件：

```
# 项目说明
- 回复使用中文
- 代码注释使用中文
- 优先使用 Rust 和 TypeScript
- 测试框架使用 proptest
```

查找顺序：`<cwd>/.baoclaw/BAOCLAW.md` → `<cwd>/BAOCLAW.md`

AI 还可以通过 `ProjectNoteTool` 自动向 BAOCLAW.md 追加发现的项目规则。

8.4 上下文窗口预算

系统提示词 + 对话历史 + 工具定义的总 token 数不能超过模型的上下文窗口。BaoClaw 的预算分配：

```
上下文窗口（200K tokens for GLM-5-Turbo）
├── 系统提示词：~5K tokens（基础 + 项目指令 + Skills + 记忆）
├── 工具定义：~3K tokens（15 个工具的 schema + description）
├── 对话历史：~190K tokens（动态，超出时自动 compact）
└── 输出预留：~2K tokens
```

8.5 小结

系统提示词是 Agent 行为的“宪法”。BaoClaw 通过分层组装（基础 → 项目 → Git → Skills → 记忆），让每次 API 调用都携带完整的上下文信息。

第 9 章：短期记忆 —— 对话历史与 Compact

9.1 问题：对话越来越长怎么办

Agent 的对话历史会不断增长。每次用户消息、AI 回复、工具调用和结果都会追加到历史中。当历史超过模型的上下文窗口时，API 调用会失败。

9.2 BaoClaw 的短期记忆

对话历史存储在 `QueryEngine.messages: Vec<Message>` 中。每条消息有类型：

```
pub enum MessageContent {
    User { message: ApiUserMessage },       // 用户消息
    Assistant { message: ApiAssistantMessage }, // AI 回复
    System { subtype, content },            // 系统消息（compact 边界）
}
```

每次 API 调用时，完整的 `messages` 列表被转换为 API 格式发送给 LLM。

9.3 Compact 机制

当对话历史太长时，`compact` 方法将旧消息压缩为摘要：

```rust
pub async fn compact(&mut self) -> Result<CompactResult, EngineError> {
    let keep_recent: usize = 4;  // 保留最近 4 条消息
    let tokens_before = estimate_tokens(&self.messages);

    let split = self.messages.len() - keep_recent;
    let old_messages = &self.messages[..split];

    // 调用 LLM 生成摘要
    let summary = self.call_api_for_summary(&summary_prompt).await?;

    // 替换：[摘要边界] + 最近 4 条消息
    self.messages = vec![boundary_message(summary)];
    self.messages.extend(recent_messages);

    let tokens_after = estimate_tokens(&self.messages);
    Ok(CompactResult { tokens_saved, summary_tokens, tokens_before, tokens_after })
}
```

自动 Compact

当 LLM 返回 `stop_reason: "model_context_window_exceeded"` 时，BaoClaw 自动触发 compact：

```rust
if stop_reason == Some("model_context_window_exceeded") {
    // 通知客户端
    tx.send(EngineEvent::AssistantChunk {
        content: "🗜 上下文窗口已满，正在自动压缩..."
    }).await;

    // 执行 compact
    // ... 生成摘要，替换旧消息 ...

    tx.send(EngineEvent::AssistantChunk {
        content: "✅ 压缩完成，正在重试..."
    }).await;

    continue; // 重试查询
}
```

图片数据剥离

MCP 截图工具返回的 base64 图片数据（5-10MB）会撑爆上下文。BaoClaw 在工具结果存入对话历史前，自动剥离图片数据：

```rust
fn strip_base64_images(value: &Value) -> Value {
    // 检测 base64 图片数据，替换为 "[image: base64 data removed]"
    // 图片已经通过 tool_result 事件实时发送给客户端了
}
```

9.4 小结

短期记忆是 Agent 的“工作记忆”。Compact 机制通过摘要压缩保持对话历史在上下文窗口内，自动 compact 确保用户不会遇到“上下文超出”的错误。

第 10 章：长期记忆 —— 跨会话的知识持久化

10.1 问题：Agent 的“失忆症”

短期记忆只存在于 daemon 进程的生命周期内。重启 daemon 后，AI 不记得你是谁、你喜欢什么、之前做过什么决定。

用户会反复告诉 AI 同样的信息：“我喜欢用中文”、“这个项目用 Rust”、“我的名字是小明”。这是糟糕的体验。

10.2 模式：JSONL 文件 + 系统提示词注入

BaoClaw 采用最简单的方案： 1. 记忆存储在 `~/.baoclaw/memory.jsonl` （一行一条记忆） 2. daemon 启动时加载所有记忆 3. 注入到系统提示词的 `# Long-term Memory` 部分

不用向量数据库、不用 RAG —— 对于 <100 条记忆，直接全量注入最简单可靠。

10.3 BaoClaw 的实现

数据模型（`claude-core/src/engine/memory.rs`）

```rust
pub struct MemoryEntry {
    pub id: String,            // 短 UUID（8 字符）
    pub content: String,       // 记忆内容
    pub category: MemoryCategory,  // fact / preference / decision
    pub created_at: String,    // ISO 8601 时间戳
    pub source: String,        // "user"（手动）或 "auto"（AI 自动）
}

pub enum MemoryCategory {
    Fact,         // 事实："用户的名字是小明"
    Preference,   // 偏好："用户喜欢用中文交流"
    Decision,     // 决策："我们决定用 Rust 重写这个模块"
}
```

存储（MemoryStore）

```
pub struct MemoryStore {
    entries: Mutex<Vec<MemoryEntry>>,
    file_path: PathBuf,  // ~/.baoclaw/memory.jsonl
}

impl MemoryStore {
    pub fn load() -> Self { /* 从磁盘读取 */ }
    pub async fn add(&self, content, category, source) -> MemoryEntry { /* 追加写入
  */ }
    pub async fn delete(&self, id_prefix: &str) -> bool { /* 删除并重写文件 */ }
    pub async fn build_prompt_fragment(&self) -> Option<String> { /* 生成系统提示词片
  段 */ }
}
```

注入到系统提示词

```
pub async fn build_prompt_fragment(&self) -> Option<String> {
    let entries = self.entries.lock().await;
    if entries.is_empty() { return None; }

    // 按分类组织
    let mut parts = vec!["# Long-term Memory\n\n..."];
    parts.push("## Facts");
    for e in facts { parts.push(format!("- {}", e.content)); }
    parts.push("## Preferences");
    for e in prefs { parts.push(format!("- {}", e.content)); }
    parts.push("## Decisions");
    for e in decisions { parts.push(format!("- {}", e.content)); }

    Some(parts.join("\n"))
}
```

10.4 两种写入方式

手动：`/memory` 命令

```
/memory add preference 我喜欢用中文交流
/memory add fact 我的项目使用 Rust + TypeScript
/memory list
/memory delete abc123
```

终端和 Telegram 都支持。

自动：MemoryTool

AI 可以主动调用 `MemoryTool` 保存发现的信息：

```
pub struct MemoryTool;

impl Tool for MemoryTool {
    fn prompt(&self) -> String {
        "Save important information to long-term memory. Use this when you discover
         user preferences, important facts, or decisions..."
    }

    async fn call(&self, input: Value, ...) -> Result<ToolResult, ToolError> {
        // 直接追加到 ~/.baoclaw/memory.jsonl
    }
}
```

AI 在对话中发现“用户总是要求用中文回复”时，会自动调用这个工具保存偏好。

10.5 小结

长期记忆让 Agent 真正“认识”用户。JSONL + 全量注入的方案虽然简单，但对于个人使用场景完全够用。当记忆量增长到上千条时，可以考虑引入向量检索做相关性过滤。

第 11 章：会话设计 —— 共享、恢复与多客户端

11.1 问题：多个入口，一个大脑

用户可能在终端写代码，同时在手机 Telegram 上查看进度。两个客户端需要看到同一个对话上下文 —— 在终端说的话，Telegram 要能接着聊。

11.2 会话模式

BaoClaw 支持两种会话模式：

独立模式（Independent）

每个客户端连接创建独立的 QueryEngine，对话历史完全隔离。这是传统模式。

```
Client A → QueryEngine A (独立历史)
Client B → QueryEngine B (独立历史)
```

共享模式（Shared）

多个客户端通过 `shared_session_id` 连接到同一个 QueryEngine。

```
Client A ─┐
          ├→ SharedSession → QueryEngine (共享历史)
Client B ─┘
```

11.3 SharedSession 的设计（`claude-core/src/engine/shared_session.rs`）

```
pub struct SharedSession {
    engine: Arc<RwLock<QueryEngine>>,            // 共享的引擎
    active_submitter: Mutex<Option<ClientId>>,   // 当前提交者锁
    event_tx: broadcast::Sender<EngineEvent>,    // 事件广播
    connected_clients: Mutex<HashSet<ClientId>>, // 已连接客户端
}
```

并发控制：ActiveSubmitter 锁

同一时间只有一个客户端能提交消息，避免对话上下文混乱：

```
pub async fn try_acquire_submitter(&self, client_id: ClientId) -> bool {
    let mut submitter = self.active_submitter.lock().await;
    if submitter.is_none() {
        *submitter = Some(client_id);
        true  // 获取成功
    } else {
        false // 已有其他客户端在提交
    }
}
```

其他客户端尝试提交时收到 `-32001 session busy` 错误。

事件广播

提交者的 QueryEngine 产生的事件通过 `broadcast::channel` 广播给所有客户端：

```
// 提交者发送事件
session.broadcast(event.clone());

// 其他客户端的后台任务接收
loop {
    match rx.recv().await {
        Ok(event) => {
            if !session.is_active_submitter(client_id).await {
                conn.send_notification("stream/event", event).await;
            }
        }
    }
}
```

提交者自己通过直接发送接收事件（不走广播），避免重复。

生命周期

```
// 客户端断开
let is_last = session.remove_client(client_id).await;
if is_last {
    // 最后一个客户端断开，清理 SharedSession
    session_registry.remove(&session_id).await;
}
```

11.4 会话持久化（Transcript）

`TranscriptWriter` 将每条消息写入 `~/.baoclaw/sessions/<session_id>.jsonl`：

```
// 每次用户消息、AI 回复、工具调用都追加到文件
append_transcript(&mut writer, &TranscriptEntry {
    timestamp: "2026-04-07T...",
    entry_type: TranscriptEntryType::UserMessage,
    data: serde_json::to_value(&msg),
});
```

会话恢复

daemon 重启后，通过 `resume_session_id` 从磁盘恢复：

```
let entries = TranscriptWriter::load(resume_id)?;
let messages = rebuild_messages_from_transcript(&entries);
engine.set_messages(messages);
```

11.5 BaoClaw 的客户端连接流程

```
CLI 启动 → initialize { shared_session_id: "default" }
                                        ↓
Daemon: SessionRegistry.get_or_create("default")
  → 不存在: 创建新 QueryEngine + SharedSession
  → 已存在: 加入现有 SharedSession
                                        ↓
返回 { shared: true, message_count: N }
                                        ↓
Telegram 启动 → initialize { shared_session_id: "default" }
                                        ↓
Daemon: 加入同一个 SharedSession
```

CLI 和 Telegram 都用 `shared_session_id: "default"`，自动共享同一个会话。

11.6 小结

会话设计是多客户端 Agent 系统的核心挑战。BaoClaw 通过 SharedSession + ActiveSubmitter 锁 + broadcast 广播，实现了安全的多客户端实时共享。

第 12 章：守护进程架构 —— Daemon 模式

12.1 问题：Agent 进程的生命周期

如果 Agent 和 CLI 在同一个进程里，CLI 退出 Agent 就死了。MCP 服务器连接断开、对话历史丢失、Telegram 无法接入。

12.2 模式：Daemon + 客户端分离

BaoClaw 将 Agent Core 作为独立的守护进程运行：

```
baoclaw (CLI)
  └→ spawn: claude-core --daemon --cwd /path/to/project
       ├→ 绑定 UDS socket
       ├→ 连接 MCP 服务器
       ├→ 加载 Skills、Memory
       └→ 等待客户端连接（accept loop）
```

启动流程

1. CLI 检查是否有运行中的 daemon（扫描 `/tmp/baoclaw-sockets/`）
2. 如果有：直接连接
3. 如果没有：spawn 新的 daemon 进程
4. Daemon 输出 `SOCKET:/tmp/baoclaw-sockets/baoclaw-xxx.sock`
5. CLI 连接到该 socket

发现机制

Daemon 在 socket 旁边写一个 metadata JSON 文件：

```json
{
    "pid": 12345,
    "cwd": "/home/user/project",
    "session_id": "abc-def-123",
    "socket": "/tmp/baoclaw-sockets/baoclaw-e102213c-12345.sock",
    "started_at": "2026-04-07T10:00:00Z"
}
```

客户端通过扫描这个目录发现可用的 daemon 实例。

进程存活检查

```
function discoverDaemons(): DaemonInfo[] {
    for (const file of fs.readdirSync(socketDir)) {
        const meta = JSON.parse(fs.readFileSync(file));
        try { process.kill(meta.pid, 0); } catch { continue; } // 进程已死
        if (!fs.existsSync(meta.socket)) continue; // socket 不存在
        daemons.push(meta);
    }
}
```

Shutdown

```
// accept loop 使用 tokio::select! 竞争 accept 和 should_exit 检查
let accept_result = tokio::select! {
    result = server.accept() => Some(result),
    _ = async {
        loop {
            tokio::time::sleep(Duration::from_millis(500)).await;
            if should_exit.load(Ordering::Relaxed) { break; }
        }
    } => None,
};
```

`/shutdown` 设置 `should_exit = true`，accept loop 在 500ms 内退出，执行 `cleanup_meta` 清理 socket 和 metadata 文件。

12.3 小结

Daemon 模式让 Agent Core 独立于任何客户端运行，是多客户端架构的基础。

第 13 章：IPC 协议 —— JSON-RPC over UDS

13.1 协议选择

BaoClaw 使用 JSON-RPC 2.0 over Unix Domain Socket（UDS），NDJSON 分帧（每行一个 JSON 消息）。

为什么不用 HTTP/WebSocket？ - UDS 是本地通信，零网络开销 - 不需要端口管理，不会端口冲突 - 文件系统权限天然提供安全隔离

13.2 消息类型

请求（Request）

```
{"jsonrpc":"2.0","id":1,"method":"submitMessage","params":{"prompt":"hello"}}
```

响应（Response）

```
{"jsonrpc":"2.0","id":1,"result":{"status":"complete"}}
```

通知（Notification）—— 无 id，无需响应

```
{"jsonrpc":"2.0","method":"stream/event","params":
  {"type":"assistant_chunk","content":"Hi"}}
```

13.3 方法列表（`claude-core/src/ipc/router.rs`）

```rust
pub enum ClientMethod {
    Initialize { cwd, model, settings, resume_session_id, shared_session_id },
    SubmitMessage { prompt },
    Abort,
    Shutdown,
    UpdateSettings { settings },
    PermissionResponse { tool_use_id, decision, rule },
    ListTools,
    ListMcpServers,
    ListSkills,
    ListPlugins,
    Compact,
    SwitchModel { model },
    GitDiff,
    GitCommit { message },
    GitStatus,
    MemoryList,
    MemoryAdd { content, category },
    MemoryDelete { id },
    MemoryClear,
    TaskCreate { description, prompt },
    TaskList,
    TaskStatus { task_id },
    TaskStop { task_id },
}
```

13.4 流式事件

Agent 的回复通过 `stream/event` 通知实时推送：

```
Client → submitMessage
Daemon → stream/event { type: "assistant_chunk", content: "你" }
Daemon → stream/event { type: "assistant_chunk", content: "好" }
Daemon → stream/event { type: "tool_use", tool_name: "Bash", ... }
Daemon → stream/event { type: "tool_result", output: "...", ... }
Daemon → stream/event { type: "result", status: "complete", ... }
Daemon → response { id: 1, result: { status: "complete" } }
```

客户端收到 `assistant_chunk` 就实时渲染，收到 `result` 就知道本轮结束。

13.5 小结

JSON-RPC 2.0 + UDS + NDJSON 是一个简单高效的本地 IPC 方案。请求-响应模式处理命令，通知模式处理流式事件。

第 14 章：多客户端接入 —— 终端、Telegram、更多

14.1 客户端是“薄”的

BaoClaw 的客户端只负责两件事：用户输入和结果展示。所有业务逻辑在 Daemon 端。

14.2 终端 CLI（`ts-ipc/cli.ts`）

富文本终端界面，功能包括： - ANSI 彩色输出、Markdown 渲染 - Spinner 动画（等待 AI 回复时） - 斜杠命令（`/tools`、`/memory`、`/compact` 等） - Tab 补全（命令和文件路径） - 权限交互（工具执行前询问用户）

命令路由

```
if (input === '/tools') { /* 调用 listTools RPC */ }
if (input === '/compact') { /* 调用 compact RPC */ }
if (input.startsWith('/memory')) { /* 调用 memory* RPC */ }
// 其他输入 → submitMessage RPC
```

14.3 Telegram 网关（`baoclaw-telegram/src/gateway.ts`）

Telegram Bot 作为另一个客户端连接到同一个 Daemon。

架构

```
Telegram API ←→ Gateway 进程 ←→ Daemon (UDS)
```

命令路由

Gateway 本地拦截斜杠命令，调用 Daemon RPC，格式化为纯文本/HTML 返回：

```
const commandHandlers = {
    '/tools':   () => handleTools(),     // listTools RPC → formatTools()
    '/skills':  () => handleSkills(),    // listSkills RPC → formatSkills()
    '/memory':  (args) => handleMemory(args), // memory* RPC
    '/shutdown': () => handleShutdown(), // shutdown RPC
    '/quit':    () => handleQuit(),      // 断开 gateway
};
```

未注册的命令和普通消息通过 `submitMessage` 转发给 AI。

消息格式化

- 文本：Markdown → Telegram HTML（`markdownToTelegramHtml`），失败降级纯文本
- 图片：base64 → 临时文件 → `bot.sendPhoto()`
- 工具调用：显示 `ϟ ToolName`
- 错误：显示 `✕ 错误信息`

Session Busy 处理

共享模式下，如果终端正在提交消息，Telegram 用户会收到友好提示：

```
if (msg.includes('session busy')) {
    bot.sendMessage(chatId, '⏳ 会话正忙，另一个客户端正在提交消息，请稍后再试。');
}
```

14.4 添加新客户端

要添加一个新客户端（比如 WhatsApp、Web UI），只需要：

1. 实现 IPC 客户端（JSON-RPC over UDS）
2. 发送 `initialize` 请求（带 `shared_session_id: "default"`）
3. 监听 `stream/event` 通知
4. 实现命令路由和消息格式化

核心逻辑全在 Daemon 端，客户端只是“皮肤”。

14.5 小结

多客户端架构的关键是 Daemon 端集中处理所有业务逻辑，客户端只负责 UI。这让添加新的接入方式变得非常简单。

第 15 章：共享会话 —— SharedQueryEngine 的设计

15.1 问题：从“各自独立”到“实时共享”

最初 BaoClaw 的每个客户端连接都创建独立的 QueryEngine。`resume_session_id` 只能在连接时从磁盘加载一次历史快照，之后两端完全隔离。

用户在终端说“小明22:00睡，小王晚一个小时”，然后在 Telegram 问“小王几点睡” —— AI 不知道，因为两个 QueryEngine 的对话历史是独立的。

15.2 设计：SessionRegistry + SharedSession

```
// 全局注册表
pub struct SessionRegistry {
    sessions: Mutex<HashMap<String, Arc<SharedSession>>>,
}

// 共享会话
pub struct SharedSession {
    engine: Arc<RwLock<QueryEngine>>,            // 共享引擎
    active_submitter: Mutex<Option<ClientId>>,   // 并发锁
    event_tx: broadcast::Sender<EngineEvent>,    // 事件广播
    connected_clients: Mutex<HashSet<ClientId>>, // 客户端集合
}
```

查找或创建

```rust
pub async fn get_or_create(&self, session_id: &str, factory: impl FnOnce() ->
  QueryEngine)
    -> (Arc<SharedSession>, bool)
{
    let mut sessions = self.sessions.lock().await;
    if let Some(existing) = sessions.get(session_id) {
        (Arc::clone(existing), false)  // 加入已有会话
    } else {
        let session = Arc::new(SharedSession::new(factory(), 256));
        sessions.insert(session_id.to_string(), Arc::clone(&session));
        (session, true)  // 创建新会话
    }
}
```

15.3 并发控制的三个层次

层次 1：ActiveSubmitter 锁

同一时间只有一个客户端能 `submitMessage`。其他客户端收到 `-32001 session busy`。

层次 2：RwLock 读写分离

- 读操作（`get_messages`、`abort`）：多个客户端可以并发
- 写操作（`submit_message`、`compact`）：独占

层次 3：操作分类

操作	有 ActiveSubmitter 时
listTools/listSkills/gitStatus	✅ 正常执行
compact/switchModel	❌ 返回 -32002
abort	✅ 任何客户端都可以中止
submitMessage	❌ 返回 -32001

15.4 事件广播

提交者产生的事件通过 `tokio::sync::broadcast` 广播给所有客户端：

```
Client A (提交者): submitMessage → QueryEngine → events
                                                    ↓
                                              broadcast channel
                                              ↙             ↘
                                        Client A          Client B
                                        (直接发送)        (广播接收)
```

提交者通过直接发送接收事件（避免重复），其他客户端通过广播接收。

15.5 小结

共享会话是多客户端 Agent 系统的核心。通过 SessionRegistry 管理会话生命周期，SharedSession 提供并发安全的 QueryEngine 访问，broadcast channel 实现实时事件同步。

第 16 章：错误处理与恢复 —— 从 Fallback 到自动 Compact

16.1 问题：生产环境中什么都会出错

API 会限流、网络会断开、上下文会超出、工具会执行失败。一个生产级 Agent 系统必须优雅地处理每一种错误。

16.2 错误分类与处理策略

API 错误

BaoClaw 在 `run_query_loop` 中处理 API 调用错误：

```
match api_client.create_message_stream(request).await {
    Ok(stream) => { /* 正常处理 */ }
    Err(ApiError::RateLimited) => {
        // 限流 → Fallback 控制器决定重试还是切换模型
        match fallback_controller.on_rate_limit() {
            FallbackAction::Retry { delay, .. } => {
                tokio::time::sleep(delay).await;
                continue;
            }
            FallbackAction::Fallback { from, to } => {
                tx.send(EngineEvent::ModelFallback { from, to }).await;
                continue;
            }
            FallbackAction::Exhausted { .. } => {
                tx.send(EngineEvent::Error(...)).await;
                return;
            }
        }
    }
    Err(e) => {
        tx.send(EngineEvent::Error(...)).await;
        return;
    }
}
```

上下文窗口超出

当 LLM 返回 `stop_reason: "model_context_window_exceeded"` 时：

```
if stop_reason == Some("model_context_window_exceeded") {
    // 1. 通知客户端
    tx.send(AssistantChunk { content: "🗜 上下文窗口已满，正在自动压缩..." }).await;

    // 2. 移除空的 assistant 回复
    messages.pop(); // empty assistant
    let user_msg = messages.pop(); // 保存用户消息

    // 3. 内联 compact：保留最近 4 条，摘要其余
    let summary = call_api_for_summary(&old_messages).await;
    messages = vec![boundary(summary)] + recent;

    // 4. 恢复用户消息
    messages.push(user_msg);

    // 5. 通知并重试
    tx.send(AssistantChunk { content: "✅ 压缩完成，正在重试..." }).await;
    continue;
}
```

用户看到的效果：AI 短暂停顿，显示压缩提示，然后正常回复。

工具执行失败

工具返回 `ToolError` 时，结果以 `is_error: true` 传回 LLM，让 AI 自己决定如何处理（重试、换方法、或告知用户）。

MCP 服务器连接失败

```
match connect_result {
    Ok(Ok(())) => { /* 注册工具 */ }
    Ok(Err(e)) => {
        eprintln!("Warning: MCP server '{}' failed: {}", name, e);
        // 跳过，不影响其他功能
    }
    Err(_) => {
        eprintln!("Warning: MCP server '{}' timed out (30s)", name);
    }
}
```

MCP 连接失败不会阻止 daemon 启动，只是该 MCP 的工具不可用。

16.3 设计原则

1. **降级而非崩溃** —— 能降级就降级，不要让一个错误杀死整个系统
2. **通知用户** —— 自动恢复时告诉用户发生了什么
3. **让 AI 决策** —— 工具失败时把错误信息传给 AI，让它决定下一步
4. **超时保护** —— MCP 连接 30s 超时，避免无限等待

16.4 小结

错误处理是生产系统的生命线。BaoClaw 通过模型 Fallback、自动 Compact、优雅降级三层防护，确保用户几乎不会遇到无法恢复的错误。

第 17 章：流式输出 —— SSE 事件与广播

17.1 问题：用户不想等

AI 生成一段长回复可能需要 10-30 秒。如果等全部生成完再显示，用户体验很差。流式输出让用户看到 AI “一个字一个字地打”。

17.2 两层流式架构

```
LLM API (SSE) → QueryEngine (mpsc) → IPC (notification) → Client (渲染)
```

第一层：LLM API → QueryEngine

LLM API 返回 SSE 流，QueryEngine 逐事件处理：

```
while let Some(event) = stream.next().await {
    match event {
        ContentBlockDelta { delta, .. } => {
            if let Some(text) = delta.get("text") {
                // 立即发送给客户端
                tx.send(EngineEvent::AssistantChunk { content: text }).await;
            }
        }
        ContentBlockStop { .. } => { /* 一个 block 结束 */ }
        MessageStop => break,
    }
}
```

第二层：QueryEngine → Client

通过 `mpsc::channel` 发送 `EngineEvent`，IPC 层转换为 JSON-RPC notification：

```
pub fn engine_event_to_notification(event: &EngineEvent) -> JsonRpcNotification {
    JsonRpcNotification::new("stream/event", serde_json::to_value(event))
}
```

17.3 事件类型（`EngineEvent`）

```
pub enum EngineEvent {
    AssistantChunk { content: String, tool_use_id: Option<String> },
    ThinkingChunk { content: String },
    ToolUse { tool_name: String, input: Value, tool_use_id: String },
    ToolResult { tool_use_id: String, output: Value, is_error: bool },
    PermissionRequest { tool_name: String, input: Value, tool_use_id: String },
    Progress { tool_use_id: String, data: Value },
    StateUpdate { patch: Value },
    Result(QueryResult),
    Error(EngineError),
    ModelFallback { from_model: String, to_model: String },
}
```

每种事件类型对应 Agent 执行过程中的一个关键时刻。客户端根据事件类型决定如何渲染。

17.4 共享模式下的广播

独立模式下，事件通过 `mpsc` 直接发送给唯一的客户端。共享模式下，需要广播给所有客户端：

```
// 提交者产生事件时
session.broadcast(event.clone());  // broadcast::Sender

// 每个客户端的后台任务
loop {
    match broadcast_rx.recv().await {
        Ok(event) => {
            // 跳过提交者自己（避免重复）
            if session.is_active_submitter(client_id).await { continue; }
            conn.send_notification("stream/event", event).await;
        }
        Err(RecvError::Lagged(n)) => { /* 客户端太慢，跳过 n 个事件 */ }
        Err(RecvError::Closed) => break,
    }
}
```

`tokio::sync::broadcast` 的 `Lagged` 错误处理很重要 —— 如果某个客户端处理太慢（比如 Telegram 网络延迟），不会阻塞其他客户端。

17.5 客户端渲染

终端 CLI

```
case 'assistant_chunk':
    currentText += content;
    // 累积文本，最后通过 Markdown 渲染器输出
    break;
case 'tool_use':
    console.log(`ϟ ${toolName}`);
    startSpinner(`Running ${toolName}...`);
    break;
```

Telegram

```
case 'assistant_chunk':
    accumulators.set(chatId, current + content);
    // 累积，等 result 事件时一次性发送
    break;
case 'result':
    const text = accumulators.get(chatId);
    bot.sendMessage(chatId, markdownToTelegramHtml(text), { parse_mode: 'HTML' });
    break;
```

Telegram 不能逐字发送（API 限制），所以累积全部文本后一次性发送。

17.6 小结

流式输出是 Agent 用户体验的关键。BaoClaw 通过 SSE → mpsc → IPC notification 的三层管道，实现了从 LLM 到客户端的实时事件流。共享模式下通过 broadcast channel 扩展到多客户端。

第 18 章：权限控制 —— PermissionGate 模式

18.1 问题：AI 不应该无限制地行动

Agent 可以执行 shell 命令、写文件、删文件。如果不加控制，一个错误的 AI 决策可能造成不可逆的损害。

18.2 模式：请求-授权-执行

BaoClaw 的权限控制采用“请求-授权-执行”三步模式：

1. AI 决定调用一个危险工具（如 `BashTool`）
2. Daemon 发送 `permission_request` 事件给客户端
3. 用户选择：允许（Allow）/ 始终允许（Always Allow）/ 拒绝（Deny）
4. Daemon 根据用户决策执行或跳过

18.3 BaoClaw 的实现

PermissionGate（`claude-core/src/permissions/gate.rs`）

```
pub struct PermissionGate {
    pending: Mutex<HashMap<String, oneshot::Sender<PermissionDecision>>>,
}

pub enum PermissionDecision {
    Allow,
    AllowAlways { rule: Option<String> },
    Deny,
}
```

工作流程：

```
// 1. QueryEngine 遇到需要授权的工具
let (tx, rx) = oneshot::channel();
permission_gate.pending.insert(tool_use_id, tx);

// 2. 发送 permission_request 事件给客户端
send_event(EngineEvent::PermissionRequest { tool_name, input, tool_use_id });

// 3. 等待用户响应
let decision = rx.await;

// 4. 根据决策执行
match decision {
    Allow | AllowAlways => execute_tool(),
    Deny => skip_tool(),
}
```

客户端交互

终端 CLI 显示权限请求：

```
⚠ Permission Required
  Tool: Bash
  Input: {"command": "rm -rf /tmp/test"}
  [y] Allow  [a] Always Allow  [n] Deny
>
```

用户输入 `y` / `a` / `n`，CLI 发送 `permissionResponse` RPC 给 Daemon。

Always Allow 规则

选择 “Always Allow” 后，该工具的后续调用自动通过，不再询问。规则存储在内存中（当前 session 有效）。

18.4 哪些工具需要授权

工具	是否需要授权	原因
BashTool	✔	可执行任意命令
FileWriteTool	✔	可覆盖任意文件
FileEditTool	✔	可修改任意文件
FileReadTool	×	只读，无副作用
GrepTool	×	只读
WebFetchTool	×	只读
MemoryTool	×	写入用户自己的记忆文件

18.5 小结

权限控制是 Agent 安全的最后一道防线。PermissionGate 通过 oneshot channel 实现异步的请求-等待-响应模式，让用户对 AI 的危险操作保持控制权。

第 19 章：成本追踪 —— Token 计量与预算

19.1 问题：AI 调用不是免费的

每次 LLM API 调用都消耗 token，token 就是钱。一个复杂的多轮工具调用可能消耗数万 token。用户需要知道花了多少钱。

19.2 BaoClaw 的成本追踪

CostTracker（`claude-core/src/engine/cost_tracker.rs`）

```
pub struct CostTracker {
    total_input_tokens: u64,
    total_output_tokens: u64,
    query_input_tokens: u64,    // 当前查询的 token
    query_output_tokens: u64,
}

impl CostTracker {
    pub fn accumulate(&mut self, usage: &Usage, model: &str) {
        self.total_input_tokens += usage.input_tokens;
        self.total_output_tokens += usage.output_tokens;
        // ... 按模型计算美元成本
    }

    pub fn total_cost(&self) -> f64 { /* 总成本 USD */ }
    pub fn current_query_cost(&self) -> f64 { /* 当前查询成本 */ }
}
```

Token 来源

每次 API 调用返回 usage 信息：

```
ApiStreamEvent::MessageStart { message } => {
    if let Some(usage) = message.get("usage") {
        cost_tracker.accumulate(&usage, &config.model);
    }
}
ApiStreamEvent::MessageDelta { usage, .. } => {
    cost_tracker.accumulate(&usage, &config.model);
}
```

展示给用户

每次查询完成后，CLI 显示 token 统计：

```
2 tools · 6141→2164 tokens · 58.5s
```

格式：`工具数 · 输入→输出 tokens · 耗时`

缓存 Token

Anthropic API 支持 prompt caching，BaoClaw 追踪缓存相关的 token：

```rust
pub struct Usage {
    pub input_tokens: u64,
    pub output_tokens: u64,
    pub cache_creation_input_tokens: Option<u64>,  // 创建缓存的 token
    pub cache_read_input_tokens: Option<u64>,      // 从缓存读取的 token
}
```

缓存命中时，`cache_read_input_tokens` 的成本远低于普通 `input_tokens`。

19.3 成本优化策略

1. **Compact** —— 压缩对话历史减少每次调用的 input tokens
2. **结果截断** —— `max_result_size_chars` 防止工具返回过大的结果
3. **图片剥离** —— `strip_base64_images` 从对话历史中移除图片数据
4. **模型选择** —— 不同模型价格差异巨大（GLM-5-Turbo 比 Claude 便宜很多）

19.4 小结

成本追踪让用户对 AI 消耗保持可见性。通过 token 计量 + 多种优化策略，BaoClaw 帮助用户在功能和成本之间找到平衡。

第 20 章：Computer Use —— 桌面控制 Agent

20.1 从代码到桌面

传统 Agent 操作的是文件和命令行。Computer Use 让 Agent 能操作图形界面 —— 截图、识别 UI 元素、点击、输入文字。

20.2 BaoClaw 的 Computer Use 实现

BaoClaw 通过两种方式实现桌面控制：

方式一：MCP 工具（computer-control-mcp）

```
// ~/.baoclaw/mcp.json
{
    "mcpServers": {
        "computer-control-mcp": {
            "command": "uvx",
            "args": ["computer-control-mcp@latest"]
        }
    }
}
```

提供 15 个工具：`take_screenshot`、`click_screen`、`type_text`、`move_mouse`、`press_keys`、`list_windows`、`activate_window` 等。

方式二：Skill + Bash（无 MCP 依赖）

```
# desktop-launch-app Skill
通过 xdotool + 截图分析的方式操作桌面:
1. xdotool 获取屏幕信息
2. Python + PIL 截图并分析像素
3. xdotool 移动鼠标并点击
```

Skill 方式不需要额外的 MCP 服务器，但需要系统安装 `xdotool` 和 `python3`。

20.3 图片处理的挑战

桌面截图是 Computer Use 的核心，但也带来了独特的工程挑战：

挑战 1：图片数据太大

2560×1600 的 PNG 截图，base64 编码后约 5-10MB。直接存入对话历史会撑爆上下文窗口。

解决：`strip_base64_images` 在存入对话历史前移除图片数据。

挑战 2：MCP 返回的图片格式

MCP 工具返回 JSON 格式的图片：

```
{"content": [{"type": "image", "data": "iVBOR...", "mimeType": "image/png"}]}
```

但这个 JSON 被序列化为字符串传输，且可能被 `truncate_if_needed` 截断导致 JSON 不完整。

解决：MCP 工具的 `max_result_size_chars` 设为 10MB。

挑战 3：Telegram 显示图片

Telegram 不能显示 base64 文本。需要： 1. 从 tool_result 中提取 base64 数据 2. 解码为二进制 3. 写入临时文件 4. 通过 `bot.sendPhoto(chatId, tmpFile)` 发送 5. 删除临时文件

```
case 'tool_result':
    let outputObj = JSON.parse(tr.output); // 可能因截断失败
    for (const item of outputObj.content) {
        if (item.type === 'image') {
            const tmpFile = path.join(os.tmpdir(), `baoclaw-
  img-${Date.now()}.png`);
            fs.writeFileSync(tmpFile, Buffer.from(item.data, 'base64'));
            await bot.sendPhoto(chatId, tmpFile);
            fs.unlinkSync(tmpFile);
        }
    }
```

20.4 工作流示例

用户说“帮我打开微信”：

```
1. AI 调用 take_screenshot → 获取桌面截图
2. AI 分析截图，找到任务栏图标位置
3. AI 调用 click_screen(x, y) → 点击微信图标
4. AI 调用 take_screenshot → 验证微信是否打开
5. AI 回复"微信已打开"
```

整个过程是 ReAct 循环的自然延伸 —— 只是工具从“读写文件”变成了“截图点击”。

20.5 小结

Computer Use 是 Agent 能力的重大扩展。通过 MCP 协议接入桌面控制工具，Agent 可以操作任何图形界面应用。工程上的主要挑战在于大图片数据的处理和跨客户端的图片传输。

第 22 章：多模型支持 —— Fallback 与模型切换

22.1 问题：不要把鸡蛋放在一个篮子里

依赖单一模型有风险：限流、宕机、价格变动。生产系统需要多模型支持。

22.2 BaoClaw 的多模型架构

配置（`~/.baoclaw/config.json`）

```
{
    "model": "GLM-5-Turbo",
    "fallback_models": ["GLM-5.1", "GLM-5"],
    "max_retries_per_model": 2
}
```

FallbackController

当主模型限流时，自动切换到备用模型：

```rust
pub enum FallbackAction {
    Retry { model: String, attempt: u32, delay: Duration },
    Fallback { from: String, to: String },
    Exhausted { models_tried: Vec<String>, total_retries: u32 },
}

impl FallbackController {
    pub fn on_rate_limit(&mut self) -> FallbackAction {
        self.retry_count += 1;
        if self.retry_count <= self.max_retries {
            // 同一模型重试（带退避延迟）
            FallbackAction::Retry { delay: backoff_delay() }
        } else if self.has_next_model() {
            // 切换到下一个模型
            self.advance_model();
            FallbackAction::Fallback { from: old, to: new }
        } else {
            // 所有模型都用完了
            FallbackAction::Exhausted { ... }
        }
    }
}
```

用户通知

模型切换时通知客户端：

```rust
FallbackAction::Fallback { from, to } => {
    tx.send(EngineEvent::ModelFallback { from_model: from, to_model: to }).await;
    continue; // 用新模型重试
}
```

手动切换

用户可以通过 `/model` 命令查看和切换模型：

```
/model              → 显示当前模型和回退链
/model GLM-5.1      → 切换到 GLM-5.1
```

22.3 兼容不同 API

BaoClaw 通过 `ANTHROPIC_BASE_URL` 环境变量支持不同的 API 端点：

```bash
export ANTHROPIC_BASE_URL=https://your-proxy.com
```

这让 BaoClaw 可以连接到任何兼容 Anthropic API 格式的服务（包括 GLM 的兼容接口）。

22.4 小结

多模型支持通过 FallbackController 实现自动降级，通过 `/model` 命令实现手动切换，通过环境变量实现 API 端点配置。这让 BaoClaw 不绑定任何特定的模型提供商。

附录 A：BaoClaw 完整架构图

系统架构

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        Client Layer                                      │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐ │
│  │  Terminal CLI │  │   Telegram      │  │  Future Clients │ │
│  │  (TypeScript) │  │   Gateway       │  │  (WhatsApp,     │ │
│  │               │  │  (TypeScript)   │  │   Web UI, ...)  │ │
│  │  - TUI 渲染   │  │  - Bot API      │  │                 │ │
│  │  - 命令路由   │  │  - HTML 格式化  │  │                 │ │
│  │  - 权限交互   │  │  - 图片发送     │  │                 │ │
│  └──────────────┘  └──────────────────┘  └──────────────────┘ │
│        │                  │                     │          │
│        └──────────────────┴─────────────────────┘          │
│                           │                                 │
│                           │                                 │
│              IPC: JSON-RPC 2.0 / Unix Domain Socket         │
│                           │                                 │
├───────────────────────────┼─────────────────────────────────┤
│                           ▼                                 │
│                     Daemon (Rust)                           │
│                                                             │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │                    SharedState                        │  │
│  │                                                       │  │
│  │  ┌────────────────┐  ┌──────────────────┐  ┌──────────────┐ │  │
│  │  │ engine_tools │  │ SessionRegistry│  │ MemoryStore│ │  │
│  │  │ (14 builtin  │  │ (shared mode) │  │ (JSONL)    │ │  │
│  │  │  + MCP tools)│  └──────────────────┘  └──────────────┘ │  │
│  │  └────────────────┘  ┌──────────────────┐  ┌──────────────┐ │  │
│  │  ┌────────────────┐  │PermissionGate │  │ TaskManager│ │  │
│  │  │ skill_prompt │  └──────────────────┘  └──────────────┘ │  │
│  │  │ (Skills +    │  ┌──────────────────┐  ┌──────────────┐ │  │
│  │  │  Memory)     │  │ StateManager  │  │ CostTracker│ │  │
│  │  └────────────────┘  └──────────────────┘  └──────────────┘ │  │
│  └──────────────────────────────────────────────────────────┘  │
│                           │                                 │
│                           ▼                                 │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │                   QueryEngine                         │  │
│  │                                                       │  │
│  │  messages: Vec<Message>  ←→  LLM API (streaming)      │  │
│  │                          ←→  Tool Executor            │  │
│  │                          ←→  TranscriptWriter         │  │
│  │                                                       │  │
│  │  ReAct Loop:                                          │  │
│  │  [思考 → 工具调用 → 观察结果 → 再思考]*  → 最终回复    │  │
│  └──────────────────────────────────────────────────────────┘  │
│                                                             │
```

```
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │                    External Services                         │   │
│   │                                                              │   │
│   │   ┌──────────────┐   ┌──────────────────────┐   ┌──────────────────┐ │   │
│   │   │ LLM API      │   │ MCP Servers          │   │ Git              │ │   │
│   │   │ (GLM-5,      │   │ (computer-ctrl,      │   │ (diff,           │ │   │
│   │   │  Claude)     │   │  custom, ...)        │   │  commit)         │ │   │
│   │   └──────────────┘   └──────────────────────┘   └──────────────────┘ │   │
│   └──────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────┘
```

文件系统布局

```
~/.baoclaw/
├── bin/claude-core          # Rust daemon 二进制
├── config.json              # 全局配置 (model, fallback_models)
├── mcp.json                 # MCP 服务器配置
├── memory.jsonl             # 长期记忆
├── skills/                  # 全局 Skills
│   ├── desktop-launch-app/SKILL.md
│   └── debate-competition/SKILL.md
├── plugins/                 # 全局 Plugins
│   └── test-plugin/
│       ├── plugin.json
│       ├── skills/
│       └── mcp.json
├── sessions/                # 会话 Transcript
│   └── <session-id>.jsonl
├── ts-ipc/                  # CLI 客户端
│   ├── cli.ts
│   └── markdownRenderer.ts
└── baoclaw-telegram/        # Telegram 网关
    └── src/
        ├── gateway.ts
        └── commands.ts

<project>/
├── BAOCLAW.md               # 项目级指令
└── .baoclaw/
    ├── mcp.json             # 项目级 MCP
    ├── skills/              # 项目级 Skills
    └── todo.json            # TODO 列表
```

附录 B：从零搭建 Agent Harness

最小可行 Agent

如果你想从零搭建一个类似 BaoClaw 的 Agent Harness，以下是最小实现路径：

第 1 步：ReAct 循环（1 天）

```
# 最简单的 ReAct 循环
while True:
    response = llm.call(messages, tools)
    if response.has_tool_calls():
        for tool_call in response.tool_calls:
            result = execute_tool(tool_call)
            messages.append(tool_result(result))
    else:
        print(response.text)
        break
```

第 2 步：工具系统（1 天）

定义 Tool 接口，实现 BashTool 和 FileReadTool。

第 3 步：Daemon 化（2 天）

把 Agent 放到后台进程，通过 UDS + JSON-RPC 通信。

第 4 步：CLI 客户端（1 天）

连接 Daemon，发送消息，渲染流式输出。

第 5 步：记忆系统（1 天）

JSONL 文件存储记忆，注入系统提示词。

第 6 步：MCP 支持（2 天）

实现 MCP stdio transport，连接外部工具服务器。

第 7 步：多客户端（2 天）

添加 Telegram 网关，实现共享会话。

总计约 10 天，你就有了一个功能完整的 Agent Harness。

技术选型建议

组件	推荐	原因
Daemon	Rust / Go	性能、并发、内存安全
CLI	TypeScript	生态丰富、readline 支持好
IPC	JSON-RPC + UDS	简单、高效、无端口冲突
存储	JSONL 文件	零依赖、易调试
MCP	stdio transport	MCP 标准，生态丰富

附录 C：与其他框架的对比

Agent Harness 框架对比

特性	BaoClaw	Claude Code	OpenClaw	Shannon	LangGraph
语言	Rust + TS	TypeScript	TypeScript	Rust + Go + Python	Python
架构	Daemon + IPC	单进程	Daemon + IPC	三层微服务	库
多客户端	✅ CLI + Telegram	❌ CLI only	✅ CLI + Telegram	✅ API	❌
共享会话	✅ SharedSession	❌	✅	✅	❌
MCP 支持	✅	✅	✅	✅	❌
Skills	✅ 系统提示词注入	❌	✅	❌	❌
长期记忆	✅ JSONL	❌	✅	✅ 向量数据库	❌
模型 Fallback	✅	❌	✅	✅	❌
自动 Compact	✅	❌	✅	❌	❌
Computer Use	✅ via MCP	✅ 内置	✅ via MCP	❌	❌
权限控制	✅ PermissionGate	✅	✅	✅ OPA	❌
成本追踪	✅	✅	✅	✅	❌
多 Agent 编排	❌	❌	❌	✅ Temporal	✅ DAG

特性	BaoClaw	Claude Code	OpenClaw	Shannon	LangGraph
沙箱隔离	✗	✗	✗	✓ WASI	✗

选型建议

- **个人开发者**：BaoClaw / Claude Code —— 轻量、快速上手
- **团队协作**：Shannon —— 企业级治理、多租户
- **快速原型**：LangGraph —— Python 生态、灵活
- **多客户端需求**：BaoClaw / OpenClaw —— Daemon 架构天然支持

BaoClaw 的独特之处

1. **Rust Daemon** —— 高性能、低内存、长时间运行稳定
2. **真正的多客户端共享** —— 终端和 Telegram 实时共享同一个对话
3. **Skill 系统** —— 零代码扩展 Agent 行为
4. **自动记忆** —— AI 主动保存用户偏好和项目规则
5. **图片处理** —— MCP 截图 → base64 剥离 → Telegram 图片发送的完整链路